

MHP Manipulator — PyDrake Implementation Notes

Cable Framework, Computed Torque, and Routing Geometry

Isaac-sim robotics notes

June 15, 2026

Contents

1	Overview	3
2	Hardware Topology	3
3	Repository File Map	4
4	General Architecture — Block Diagrams	4
4.1	Plant A cable framework (closed loop)	4
4.2	CT-only stack (simplified)	4
5	Control Equations	5
5.1	Layer 1 — Computed torque	5
5.2	Layer 2 — Cable wrench map	5
5.3	Optional tension inner loop	5
5.4	Elbow MIT feed-forward modes	5
5.5	Layer 3 — MIT motor law	5
6	In-Depth Signal Flow (One Timestep)	6
7	Full State Diagram	6
8	Script Reference and CLI Examples	7
8.1	script_mhp_manipulator_cable_framework_pydrake.py	7
8.2	script_mhp_manipulator_ct_pydrake.py	7
8.3	cable/test_mhp_cable_routing_actual_viz.py	8
9	Key Code Snippets	8
9.1	Drake diagram wiring (cable framework)	8
9.2	Antagonistic tension split	8
9.3	Elbow ff from cable command	8
9.4	MIT motor step	8
9.5	CT-only direct actuation	9
10	Cable Routing Modules (cable/*_mhp.py)	9
11	CT-Only vs. Cable Framework	9
12	Real Hardware Path (Future)	9

13 Quick Reference Card**10**

1 Overview

The **MHP** (manipulator hybrid planar) is the *real* 2-DOF planar manipulator exported from Onshape as `manipulator_hybrid_planar_fusion`. Unlike the legacy cup-manipulator digital-twin scripts (SEA springs, exosuit co-contraction), the real MHP has:

- **Shoulder** (`jt_upper_base`, q_1): direct-drive MIT motor.
- **Elbow** (`jt_lower_upper`, q_2): **one** motor, two **antagonistic** cables (lower $+Y$ / upper $-Y$ on the same spool).
- **No series elastic springs** in the transmission path.

Two PyDrake entry scripts implement trajectory tracking:

Table 1: MHP simulation entry points

Script	Stack	Purpose
<code>script_mhp_manipulator_cable_framework_pydrake.py</code>	CT $\rightarrow$ cable framework $\rightarrow$ MIT $\rightarrow$ Plant A	Hardware-in-the-loop
<code>script_mhp_manipulator_ct_pydrake.py</code>	CT $\rightarrow$ ideal joint torques	Algorithmic
<code>cable/test_mhp_cable_routing_actual_viz.py</code>	FK + analytical cable paths	State visualization

Conda environment: `pydrake.cursor`. Simulation rate: $f_s = 500$ Hz ($\Delta t = 2$ ms).

2 Hardware Topology

Physical actuation map

$$\begin{aligned}
 \text{Shoulder: } & \tau_1 \text{ [Nm]} \xrightarrow{\text{direct}} q_1 \\
 \text{Elbow: } & F_{\text{net}} = \tau_2 / r_p \text{ [N]} \\
 & T_{\text{lower}} = \max(F_{\text{net}}, 0) \quad (+Y) \\
 & T_{\text{upper}} = \max(-F_{\text{net}}, 0) \quad (-Y) \\
 & \tau_2 = r_p (T_{\text{lower}} - T_{\text{upper}})
 \end{aligned}$$

Elbow pitch radius: $r_p = 39.4$ mm (roller OD 78.8 mm). At any instant $T_{\text{lower}} \cdot T_{\text{upper}} = 0$ (one slack).

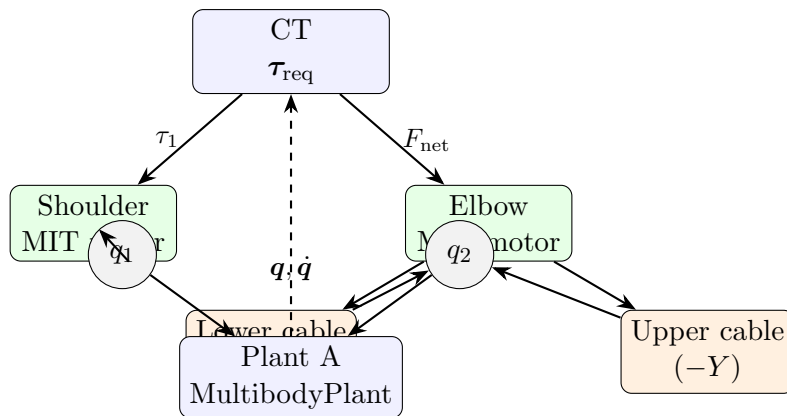


Figure 1: Hardware topology: shoulder direct-drive; elbow one motor, two antagonistic cables.

3 Repository File Map

Path	Role
<code>script_mhp_manipulator_cable_framework.py</code>	Drake entry — Plant A cable stack
<code>script_mhp_manipulator_ct_pydrake.py</code>	CT entry — CT-only (ideal actuators)
<code>cable/simulation_mhp_cable_framework.py</code>	Drake diagram build, sim loop, logging, plots
<code>cable/simulation_mhp_ct.py</code>	CT-only Drake diagram + cable viz overlay
<code>controller/mhp_ct_controller.py</code>	Alias to <code>ComputedTorqueController</code>
<code>controller/controller.py</code>	Core CT: IK, Jacobian FF, inverse dynamics
<code>controller/mhp_cable_framework.py</code>	MHPCableFramework LeafSystem
<code>controller/cable_wrench_mhp.py</code>	$\tau \rightarrow F_{\text{net}} \rightarrow T_{\text{lower}}/T_{\text{upper}}$
<code>actuators/dummy_mit_motor.py</code>	Simulated CubeMars MIT-mode motor
<code>robots/mhp_manipulator.py</code>	Drake wrapper for MHP URDF
<code>configs/robot/mhp_configs.py</code>	URDF path, joint damping/stiffness
<code>cable/*_mhp.py</code>	Analytical cable routing geometry + viz
<code>cable/test_mhp_cable_routing_actuator_viz.py</code>	Interactive Meshcat / Matplotlib routing demo
<code>cable/test_elbow_ff_from_cable.py</code>	Unit tests for elbow τ_{ff} modes

URDF: `model_using_onshape_to_robot/manipulator_hybrid_planar_fusion/manipulator_hybrid_planar_fusion_obj.urdf`.

4 General Architecture — Block Diagrams

4.1 Plant A cable framework (closed loop)

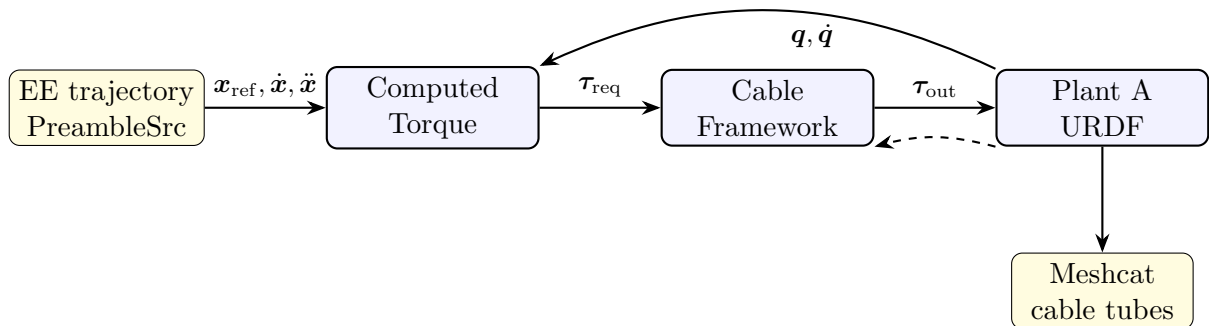


Figure 2: Drake closed-loop diagram for `simulation_mhp_cable_framework.py`.

4.2 CT-only stack (simplified)

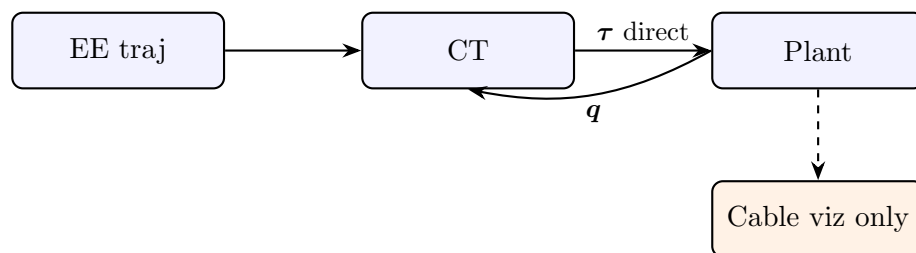


Figure 3: CT-only: torques bypass the cable/MIT layer. Cables are visualization only.

5 Control Equations

5.1 Layer 1 — Computed torque

Given end-effector reference $(\mathbf{x}_{\text{ref}}, \dot{\mathbf{x}}_{\text{ref}}, \ddot{\mathbf{x}}_{\text{ref}})$:

$$\mathbf{q}_{\text{des}} = \text{IK}(\mathbf{x}_{\text{ref}}) \quad (1)$$

$$\dot{\mathbf{q}}_{\text{ref}} = \mathbf{J}^{-1} \dot{\mathbf{x}}_{\text{ref}}, \quad \ddot{\mathbf{q}}_{\text{ref}} = \mathbf{J}^{-1}(\ddot{\mathbf{x}}_{\text{ref}} - \dot{\mathbf{J}} \dot{\mathbf{q}}) \quad (2)$$

$$\mathbf{a}_{\text{des}} = \ddot{\mathbf{q}}_{\text{ref}} + \mathbf{K}_p(\mathbf{q}_{\text{des}} - \mathbf{q}) + \mathbf{K}_d(\dot{\mathbf{q}}_{\text{ref}} - \dot{\mathbf{q}}) \quad (3)$$

$$\boldsymbol{\tau}_{\text{req}} = \mathbf{M}(\mathbf{q}) \mathbf{a}_{\text{des}} + \mathbf{h}(\mathbf{q}, \dot{\mathbf{q}}) \quad (4)$$

Default gains: $K_{p,i} = 400 \text{ s}^{-2}$, $K_{d,i} = 40 \text{ s}^{-1}$, $\tau_{\text{max}} = 10 \text{ Nm}$.

5.2 Layer 2 — Cable wrench map

From `controller/cable_wrench_mhp.py`:

$$F_{\text{net}} = \frac{\tau_{2,\text{req}}}{r_p}, \quad T_{\text{lower}} = \max(F_{\text{net}}, 0), \quad T_{\text{upper}} = \max(-F_{\text{net}}, 0)$$

$$\mathbf{W}_{\text{eff}} = \begin{bmatrix} 1 & 0 \\ 0 & r_p \end{bmatrix}, \quad \begin{bmatrix} \tau_1 \\ \tau_2 \end{bmatrix} \approx \mathbf{W}_{\text{eff}} \begin{bmatrix} \tau_{m1} \\ F_{\text{net}} \end{bmatrix}$$

5.3 Optional tension inner loop

When `--tension-kp` $K_F > 0$:

$$F_{\text{meas}} = T_{\text{lo,meas}} - T_{\text{up,meas}}, \quad F_{\text{cmd}} = F_{\text{net}} + K_F (F_{\text{net}} - F_{\text{meas}})$$

Then re-split: $T_{\text{lower,cmd}} = \max(F_{\text{cmd}}, 0)$, $T_{\text{upper,cmd}} = \max(-F_{\text{cmd}}, 0)$.

5.4 Elbow MIT feed-forward modes

Table 3: Elbow $\tau_{\text{ff},2}$ sources (`--elbow-ff-from-cable`, default ON)

Mode	Elbow MIT feed-forward
Default (<code>--elbow-ff-from-cable</code>)	$\tau_{\text{ff},2} = r_p F_{\text{cmd}}$
Legacy (<code>--no-elbow-ff-from-cable</code>)	$\tau_{\text{ff},2} = \tau_{2,\text{req}}$ (raw CT)

5.5 Layer 3 — MIT motor law

From `actuators/dummy_mit_motor.py`:

$$\tau_m = K_p(p_{\text{des}} - p) + K_d(v_{\text{des}} - v) + \tau_{\text{ff}}$$

Algebraic mode (default): $\tau_{\text{out}} = \tau_m$ immediately. Optional rotor dynamics: $J_m \ddot{\theta}_m = \tau_m - b_m \dot{\theta}_m$.

6 In-Depth Signal Flow (One Timestep)

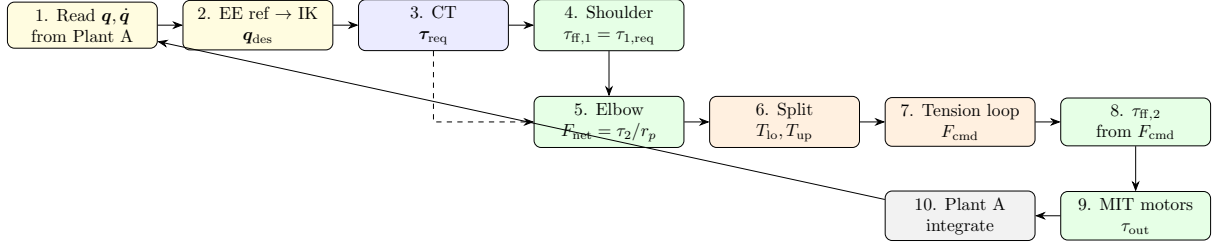


Figure 4: Per-timestep control cycle inside `MHPCableFramework.solve()` (500 Hz).

7 Full State Diagram

The diagram below shows *all signals and state variables* carried through one simulation tick. Plant state closes the loop; cable routing geometry is a parallel visualization path (not in the force loop).

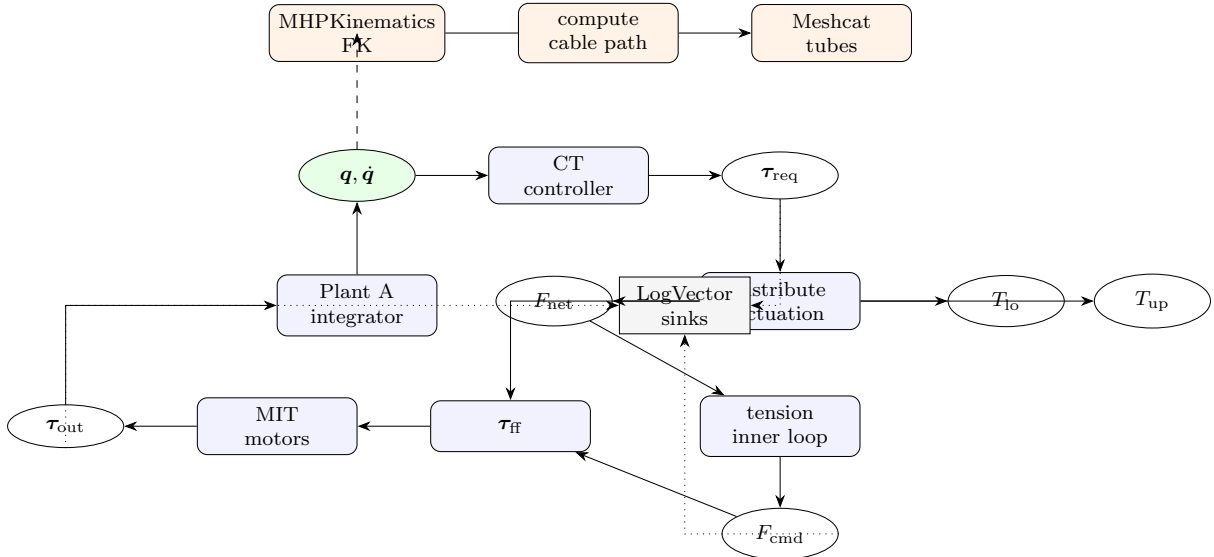


Figure 5: Full state/signal diagram: control loop (solid) vs. cable viz branch (orange, dashed).

Logged signals (cable framework run):

- `log.state` — full multibody state
- `log.tau_req` — CT raw torques
- `log.tau_out` — MIT output torques
- `log.tensions`, `log.t_meas` — commanded / measured cable tensions
- `log.tau_ff` — MIT feed-forward $[\tau_{ff,1}, \tau_{ff,2}]$
- `log.cable_cmd` — $[F_{net}, F_{cmd}, \tau_{ff,2}]$
- `log.qdes`, `log_ref`, `log.diag`

Plots saved to `plots/mhp_cable_fw_{traj}_{timestamp}.png`.

8 Script Reference and CLI Examples

8.1 `script_mhp_manipulator_cable_framework_pydrake.py`

Table 4: Cable framework CLI arguments (defaults)

Group	Argument	Default
CT	--ct-kp, --ct-kd	[400, 400], [40, 40]
CT	--ct-tau-max	10 Nm
MIT	--mit-kp, --mit-kd	[30, 15], [1.5, 0.5]
MIT	--mit-dynamics	off
Tension	--tension-kp	0.5
Tension	--tension-noise	0 N
Elbow FF	--elbow-ff-from-cable	ON
Sim	--duration, --move-duration	10 s, 3 s
Traj	--traj-type	rect
Viz	--no-meshcat, --no-show	Meshcat on, plots show

Examples:

Listing 1: Cable framework — common invocations

```

1 # Default: cable-consistent elbow ff, Meshcat + plots
2 python script_mhp_manipulator_cable_framework_pydrake.py
3
4 # Headless short run
5 python script_mhp_manipulator_cable_framework_pydrake.py \
6     --no-meshcat --no-show --duration 4 --move-duration 0
7
8 # Legacy: elbow ff tracks raw CT torque
9 python script_mhp_manipulator_cable_framework_pydrake.py \
10     --no-elbow-ff-from-cable
11
12 # Circle trajectory with stronger tension loop
13 python script_mhp_manipulator_cable_framework_pydrake.py \
14     --traj-type circle --tension-kp 1.0 --duration 8
15
16 # Rotor dynamics on MIT motors
17 python script_mhp_manipulator_cable_framework_pydrake.py \
18     --mit-dynamics --mit-kp 40 20 --mit-kd 2 1

```

8.2 `script_mhp_manipulator_ct_pydrake.py`

Same CT, simulation, robot, and trajectory groups — **no** MIT/tension/elbow-ff arguments (cables are viz-only).

Listing 2: CT-only examples

```

1 python script_mhp_manipulator_ct_pydrake.py
2 python script_mhp_manipulator_ct_pydrake.py --traj-type circle --
3     duration 8
4 python script_mhp_manipulator_ct_pydrake.py --ct-kp 400 400 --ct-kd 40
5     40 --no-show

```

8.3 cable/test_mhp_cable_routing_actual_viz.py

Listing 3: Cable routing visualization

```

1 python cable/test_mhp_cable_routing_actual_viz.py
2 python cable/test_mhp_cable_routing_actual_viz.py --q1 30 --q2 -20
3 python cable/test_mhp_cable_routing_actual_viz.py --no-matplotlib

```

Interactive Meshcat loop: enter q1 q2 in degrees to update pose.

9 Key Code Snippets

9.1 Drake diagram wiring (cable framework)

Listing 4: cable/simulation_mhp_cable_framework.py (connections)

```

1 builder.Connect(ct.GetOutputPort("actuation"), cable_fw.GetInputPort("
   tau_req"))
2 builder.Connect(plant_a.get_state_output_port(), cable_fw.GetInputPort(
   "plant_state"))
3 builder.Connect(cable_fw.GetOutputPort("actuation"), plant_a.
   get_actuation_input_port())

```

9.2 Antagonistic tension split

Listing 5: controller/cable_wrench_mhp.py

```

1 F_net = float(tau_req[1] / r_p)
2 T_lower = max(F_net, 0.0)      # +Y side
3 T_upper = max(-F_net, 0.0)    # -Y side
4 tau_elbow_ff = float(tau_req[1]) # default CT path

```

9.3 Elbow ff from cable command

Listing 6: controller/mhp_cable_framework.py (_solve)

```

1 if self._elbow_ff_from_cable:
2     tau_elbow_ff = elbow_torque_from_cable_command(F_cmd, self.
       _wrench_cfg.r_elbow)
3 else:
4     tau_elbow_ff = dist["tau_elbow_ff"]
5
6 tau_ff = tensions_to_mit_feedforward(dist["tau_shoulder_ff"],
   tau_elbow_ff)

```

9.4 MIT motor step

Listing 7: actuators/dummy_mit_motor.py

```

1 tau_m = kp * (p_des - p) + kd * (v_des - v) + tau_ff

```

9.5 CT-only direct actuation

Listing 8: cable/simulation_mhp.ct.py

```
1 builder.Connect(ct.GetOutputPort("actuation"), plant.  
  get_actuation_input_port())
```

10 Cable Routing Modules (cable/*_mhp.py)

Routing geometry is **decoupled** from the control loop. The sim updates Meshcat cable tubes from (q_1, q_2) via analytical FK — no Drake cable force elements.

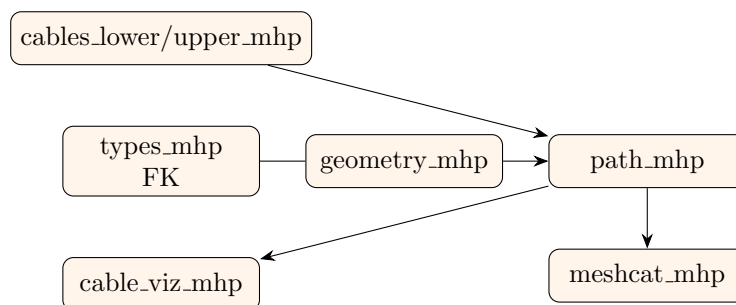


Figure 6: Cable routing module pipeline (visualization only).

11 CT-Only vs. Cable Framework

Table 5: Comparison of the two MHP simulation stacks

Aspect	CT-only	Cable framework
Control chain	Traj $\rightarrow$ CT $\rightarrow$ plant	Traj $\rightarrow$ CT $\rightarrow$ framework $\rightarrow$ plant
Shoulder actuation	Ideal τ_1	Direct-drive MIT
Elbow actuation	Ideal τ_2	One motor + antagonistic cables
Cable in force loop	No	Yes (F_{net} , tension loop)
Extra CLI	—	<code>--mit-*</code> , <code>--tension-*</code> , <code>--elbow-ff-*</code>
Log channels	5	10
Use case	Trajectory / CT tuning	Sim-to-real / hardware path

title=Not the legacy cup/exo stack

The cup-manipulator scripts (`script_cup_manipulator_pendulam_tendon_with_exo_pydrake.py`, SEA spring models, exosuit co-contraction) are **digital-twin only** and do not represent the real MHP hardware. Do not wire SEACableActuator or high- k_s spring hacks for MHP.

12 Real Hardware Path (Future)

The dummy MIT motors map directly to CubeMars drivers:

- `drivers/cubemars/motor.py` — CAN MIT mode
- Shoulder: send $\tau_{\text{ff},1}$ as MIT feed-forward
- Elbow: send $\tau_{\text{ff},2} = r_p F_{\text{cmd}}$; load cells on taut strand close the tension inner loop at the same rate as simulation (500 Hz target)

13 Quick Reference Card

One-line summary: CT plans $[\tau_1, \tau_2]$; the cable framework sends τ_1 straight to the shoulder MIT motor and maps τ_2 to antagonistic cable tensions via $F_{\text{net}} = \tau_2 / r_p$; one elbow motor drives both cables (one taut); Plant A integrates τ_{out} and feeds $\mathbf{q}, \dot{\mathbf{q}}$ back to CT.